

Aula 04 — Spring Framework

PUT ã Regra de negócio ã Vendas

Mauricio Duailibi Neto

Turma Java Entra21

Como será a aula de hoje

Parte 1 — 18h30 às 20h

- Revisão da Aula 03
- PUT: alterar um cadastro
- Buscar um item pelo id
- Retirar quantidade, com regra
- Projeto **laboratorio** — fazemos juntos

Parte 2 — 20h20 às 22h

- Voltamos ao **StockSales**
- PUT de produto, clientes e venda

Dois projetos nesta aula

Não vamos misturar os códigos.

laboratorio Primeira parte da aula. Aqui aprendemos PUT e a regra de retirar quantidade.

StockSales O sistema da loja, que já existe. Só depois do intervalo.

Quando aparecer um código, o slide vai dizer:

qual projeto e **qual arquivo** estamos mexendo.

Mapa da aula

- 1 Revisão da Aula 03
- 2 Projeto laboratorio
- 3 PUT
- 4 Buscar um pelo id
- 5 Retirar quantidade
- 6 Desafios
- 7 Parte 2 — StockSales
- 8 Recapitulando

O que já sabemos

- A lista fica **na classe** do Controller
- GET devolve a lista
- POST lê o JSON do corpo e adiciona na lista
- DELETE usa o id no endereço e remove o item
- O JavaScript chama a API com Axios

Faltou o PUT: **alterar** um cadastro que já existe.

Os quatro verbos

Ação	Verbo HTTP	O que faz
Listar / ler	GET	Lê, não muda nada
Cadastrar	POST	Cria um item novo
Editar	PUT	Troca os dados de um item
Excluir	DELETE	Apaga um item

CRUD continua o mesmo nome:

Create, Read, Update, Delete.

O Update é o PUT de hoje.

O id na URL

DELETE já usou o id no endereço:

```
DELETE /api/materiais/2
```

O 2 é o `@PathVariable`.

O método recebe esse número e procura na lista.

PUT usa o mesmo jeito:

```
PUT /api/materiais/2
```

Só que, além do id, vem um **corpo** com os dados novos.

O projeto laboratorio

A partir de hoje os exercícios da primeira parte ficam no projeto **laboratorio**.

Ele já está no repositório.

Não vamos gerar outro projeto no `start.spring.io`.

O que já vem pronto

Cadastro de materiais com GET, POST e DELETE.

Lista na classe. Formulário na tela. Botão Excluir.

Passo 1 — Atualizar o repositório

No terminal, dentro da pasta do repositório:

```
git pull
```

Deve aparecer a pasta laboratorio/.

Abram essa pasta no VS Code.

Passo 2 — Subir o Spring

Onde estamos

Projeto **laboratorio**

No terminal, dentro da pasta laboratorio:

```
./mvnw spring-boot:run
```

No Windows: `.\mvnw.cmd spring-boot:run`

Abram no navegador: `http://localhost:8080`

Um Spring por vez

Se outro projeto ainda estiver na porta 8080, parem o outro antes.

Quais arquivos já existem

Projeto: **laboratorio**

Arquivo	Para quê
Material.java	Os dados de um material
MaterialController.java	GET, POST e DELETE
static/index.html	A página
static/js/app.js	O JavaScript da página

Hoje vamos **acrescentar** métodos e botões.

Não criamos outro projeto.

Onde cada arquivo fica

```
laboratorio/  
  src/main/java/com/entra21/laboratorio/  
    LaboratorioApplication.java  
    model/Material.java  
    controller/MaterialController.java  
  src/main/resources/static/  
    index.html  
    js/app.js
```

Java fica em java/.

Página e JS ficam em resources/static/.

Conferindo o que já funciona

A página abre em `http://localhost:8080`

Caneta, Caderno e Grampeador aparecem

Cadastrar um material novo atualiza a lista

Excluir some da lista

Recarregar a página mantém o que estava na memória

Se isso não estiver ok, não sigam para o PUT.

A lista precisa estar na classe, como na aula passada.

Material.java — o que a classe guarda

Onde estamos

model/Material.java

Três dados: id, nome e quantidade.

```
public class Material {  
    private int id;  
    private String nome;  
    private int quantidade;  
}
```

Já existem construtor vazio, construtor cheio, gets e sets.

Não precisamos mexer nesta classe agora.

Cadastrar e alterar não são a mesma ação

- POST cria um material **novo** e gera um id
- PUT pega um material **que já existe** e troca nome ou quantidade

Se usarmos POST para editar, nasce um segundo cadastro.

O antigo continua lá.

Por isso o PUT leva o id: o servidor precisa saber **qual** linha da lista vai mudar.

O pedido de alteração

Exemplo: a Caneta passou a ter quantidade 20.

Pedido

- Verbo: PUT
- Endereço: /api/materiais/1
- Corpo:

```
{"nome": "Caneta", "quantidade": 20}
```

O id vai na URL.

Nome e quantidade novos vão no corpo.

O que o PUT faz, em ordem

- 1 Lê o id da URL
- 2 Lê nome e quantidade do corpo
- 3 Percorre a lista
- 4 Quando o id bater: chama `setNome` e `setQuantidade`
- 5 Devolve o material já atualizado

A lista é a mesma.

Só os campos daquele objeto mudam.

Onde estamos

Projeto **laboratorio**

Arquivo controller/MaterialController.java
método novo, dentro da classe

Imports novos: PutMapping.

PathVariable e RequestBody vocês já usam.

Onde estamos

MaterialController.java

```
@PutMapping("/api/materiais/{id}")
public Material atualizar(@PathVariable int id,
                        @RequestBody Material dados) {
    for (int i = 0; i < materiais.size(); i++) {
        if (materiais.get(i).getId() == id) {
            materiais.get(i).setNome(dados.getNome());
            materiais.get(i).setQuantidade(dados.getQuantidade());
            return materiais.get(i);
        }
    }
    return null;
}
```

Olhando o for com calma

- 1 `i` começa em 0 e anda até o fim da lista
- 2 `materiais.get(i)` é o material daquela posição
- 3 Se o id for o que veio na URL, achamos o certo
- 4 `setNome` e `setQuantidade` gravam o que veio no JSON
- 5 `return` devolve esse material e encerra o método

Se o for terminar sem achar, devolvemos `null`.

POST cria. PUT não cria

No POST vocês faziam:

```
material.setId(proximoId); e materiais.add(...)
```

No PUT **não** geramos id novo e **não** damos add.

O objeto já está na lista. Só mudamos os campos.

Se derem add no PUT, o material antigo continua e ainda nasce um duplicado.

Onde estamos

Projeto **laboratorio**

Arquivo `static/js/app.js`

Objetivo: clicar em Editar, o formulário se preenche, e o envio passa a ser um PUT.

Uma variável para o id em edição

Onde estamos

static/js/app.js, no topo do arquivo

Se estiver vazia, o form cadastra.

Se tiver um número, o form altera.

```
var idEditando = "";
```

Vamos usar essa variável em dois lugares:

- no botão Editar, para guardar o id
- no submit do form, para escolher POST ou PUT

Botão Editar dentro do for

Onde estamos

static/js/app.js, no for de carregarMateriais

```
var btnEditar = document.createElement("button");
btnEditar.textContent = "Editar";
btnEditar.setAttribute("data-id", material.id);
btnEditar.setAttribute("data-nome", material.nome);
btnEditar.setAttribute("data-quantidade", material.quantidade);

btnEditar.addEventListener("click", function (evento) {
    idEditando = evento.target.getAttribute("data-id");
    document.getElementById("nome").value =
        evento.target.getAttribute("data-nome");
    document.getElementById("quantidade").value =
        evento.target.getAttribute("data-quantidade");
});
li.appendChild(btnEditar);
```

O botão Editar fica **antes** do botão Excluir.

Por que `data-id` e `evento.target`

O `for` roda rápido e termina.

O clique acontece depois.

Se o clique usar a variável `material` do `for`, todos os botões podem pegar o último da lista.

Por isso o `id` vai para o atributo do botão, e o clique lê `evento.target`.

O submit — montar os dados

Onde estamos

static/js/app.js, no submit do form

Troquem o `axios.post` único por este bloco.

```
var dados = {  
  nome: document.getElementById("nome").value,  
  quantidade: Number(  
    document.getElementById("quantidade").value  
  )  
};
```

A quantidade vai com `Number`, porque o input devolve texto.

O submit — POST ou PUT

Onde estamos

Ainda no submit

```
if (idEditando === "") {
  axios.post("/api/materiais", dados).then(function () {
    document.getElementById("nome").value = "";
    document.getElementById("quantidade").value = "";
    carregarMateriais();
  });
} else {
  axios.put("/api/materiais/" + idEditando, dados)
    .then(function () {
      idEditando = "";
      document.getElementById("nome").value = "";
      document.getElementById("quantidade").value = "";
      carregarMateriais();
    });
}
```

Depois do PUT, idEditando volta para vazio.

Quantidade é número

O input devolve texto.

No Java o campo é `int`.

Por isso: `Number(...)`.

Sem isso, o PUT pode gravar errado ou o terminal mostra erro ao converter.

O caminho da edição

- 1 A pessoa clica em Editar na Caneta
- 2 O form mostra o nome e a quantidade atuais
- 3 Ela troca a quantidade e envia
- 4 O JS manda PUT /api/materiais/1 com JSON
- 5 O Controller acha o id 1 e atualiza
- 6 O JS chama carregarMateriais de novo

No F12, aba Network, deve aparecer um PUT.

Conferindo o PUT

Editar preenche o formulário

Enviar **não** cria um material duplicado

A quantidade nova aparece na lista

Depois de salvar, o próximo envio volta a cadastrar

No Network, o pedido é PUT, não POST

Se nascer um item a mais, o submit ainda está só no POST, ou o PUT está dando add na lista.

Lista inteira e um só material

GET /api/materiais devolve **todos**.

Às vezes precisamos de **um**:

- para ver só a Caneta
- para conferir a quantidade antes de retirar

O endereço leva o id:

GET /api/materiais/1

GET por id

Onde estamos

Projeto **laboratorio**

MaterialController.java, método novo

```
@GetMapping("/api/materiais/{id}")
public Material buscar(@PathVariable int id) {
    for (int i = 0; i < materiais.size(); i++) {
        if (materiais.get(i).getId() == id) {
            return materiais.get(i);
        }
    }
    return null;
}
```

O for é o mesmo do PUT e do DELETE.

Só muda o que fazemos quando achamos: devolver, em vez de alterar ou apagar.

Dois GET no mesmo Controller

O Spring escolhe pelo endereço:

Pedido	Método
GET /api/materiais	listar
GET /api/materiais/2	buscar

O {id} é o que diferencia os dois.

Teste no navegador

Salvem, reiniciem se precisar, e abram:

`http://localhost:8080/api/materiais/2`

Deve aparecer só o Caderno, em JSON.

`/api/materiais` continua mostrando a lista toda.

Um id que não existe devolve página vazia ou `null`.

Uma ação pode mudar um campo

Até agora o cadastro só cria, lê, troca tudo ou apaga.

Retirar é outra ação: a pessoa pede uma quantidade, e o servidor **diminui** o que está guardado.

Exemplo: Caderno tem 5.

Retirar 2 → fica 3.

A regra

Antes de diminuir, conferimos se dá.

- 1 Achar o material pelo id
- 2 Se a quantidade guardada for **menor** do que a pedida: **não muda nada**
- 3 Se der: subtrai e devolve o material

Não fica negativo

Pedir 10 de um item que tem 2 não pode passar de 2 para -8 . O servidor recusa.

O pedido de retirar

Pedido

- Verbo: POST
- Endereço: /api/materiais/2/retirar
- Corpo:

```
{"quantidade": 2}
```

POST porque é uma ação, não uma troca completa do cadastro.
O id continua na URL.

Quando não pode, o servidor recusa

Se a quantidade não alcança, não basta devolver o material antigo em silêncio. A tela precisa avisar.

Vamos lançar um erro HTTP 400:

`ResponseStatusException`

O Axios, nesse caso, cai no `catch`.

Ali mostramos a mensagem na página.

Onde estamos

Projeto **laboratorio**

MaterialController.java

Imports novos:

HttpStatus e ResponseStatusException.

```
import org.springframework.http.HttpStatus;  
import org.springframework.web.server.ResponseStatusException;
```

Onde estamos

MaterialController.java

```
@PostMapping("/api/materiais/{id}/retirar")
public Material retirar(@PathVariable int id,
                       @RequestBody Material pedido) {
    for (int i = 0; i < materiais.size(); i++) {
        Material material = materiais.get(i);
        if (material.getId() == id) {
            if (material.getQuantidade() < pedido.getQuantidade()) {
                throw new ResponseStatusException(
                    HttpStatus.BAD_REQUEST, "Quantidade insuficiente");
            }
            material.setQuantidade(
                material.getQuantidade() - pedido.getQuantidade());
            return material;
        }
    }
    return null;
}
```


O JSON do retirar

O corpo só precisa da quantidade:

```
{"quantidade": 2}
```

O Spring monta um `Material` e preenche só esse campo. O resto fica vazio.

Nós usamos apenas `pedido.getQuantidade()`.

Não damos add. Não geramos id. Não trocamos o nome.

O if da regra

```
material.getQuantidade() < pedido.getQuantidade()
```

Se for verdade: lança a exceção e **para**.

A linha do setQuantidade não roda.

Se for falso: subtrai e devolve o material.

Botão Retirar no JS

Onde estamos

static/js/app.js, no for de carregarMateriais

```
var btnRetirar = document.createElement("button");
btnRetirar.textContent = "Retirar 1";
btnRetirar.setAttribute("data-id", material.id);

btnRetirar.addEventListener("click", function (evento) {
  var id = evento.target.getAttribute("data-id");
  axios.post("/api/materiais/" + id + "/retirar", { quantidade: 1 })
    .then(function () {
      document.getElementById("mensagem").textContent = "";
      carregarMateriais();
    })
    .catch(function () {
      document.getElementById("mensagem").textContent =
        "Quantidade insuficiente";
    });
});
li.appendChild(btnRetirar);
```

then e catch

- `then` — o servidor aceitou. Atualizamos a lista.
- `catch` — o servidor recusou. Mostramos o texto no `<p id="mensagem">`.

Esse `<p>` já está no `index.html`.

Sem o `catch`, a recusa aparece só no F12 e a pessoa não entende o que aconteceu.

O caminho da retirada

- 1 Clique em Retirar 1 no Grampeador
- 2 JS manda POST com `{ quantidade: 1 }`
- 3 Controller acha o id, confere, diminui
- 4 A lista recarrega com a quantidade nova
- 5 Se já estiver em 0, o próximo clique cai no catch

No Network: um POST para `.../retirar`.

Se recusar, o status é 400.

Conferindo a retirada

Grampeador começa com 2

Dois cliques em Retirar 1 deixam quantidade 0

O terceiro clique mostra “Quantidade insuficiente”

A quantidade **não** fica negativa

Caneta e Caderno não mudam nesse teste

Erros que mais aparecem

- Porta 8080 ocupada
- PUT cria duplicado — ainda está no POST, ou o Controller dá add
- Editar preenche sempre o último — faltou `evento.target`
- Quantidade vira texto — faltou `Number`
- Retirar não avisa na tela — faltou `catch`
- Import de `ResponseStatusException` errado

Agora é com vocês

Continuem no projeto **laboratorio**.
Mesmos arquivos.

- 1 Retirar a quantidade digitada
- 2 Não cadastrar se o nome estiver vazio
- 3 No Editar, buscar o material pelo GET do id

Salve → reinicie se precisar → teste no navegador.
F12, aba Network.

Desafio 1 — quantidade da retirada

Onde estamos

`index.html` e `static/js/app.js`

Hoje o botão sempre manda 1.

Coloquem um input na página, por exemplo `id="qtd-retirar"`.

No clique, leiam esse valor com `Number` e mandem no JSON no lugar do 1.

Testem retirar 3 de um item que tem 2:
tem que recusar.

Desafio 2 — nome vazio

Onde estamos

`static/js/app.js`, no submit

Antes do Axios, se o nome estiver vazio, mostrem uma mensagem e deem return.

O POST nem chega a sair.

A lista permanece igual.

Desafio 3 — Editar busca pelo id

Onde estamos

static/js/app.js

No clique de Editar, em vez de copiar nome e quantidade dos data-..., chamem:

```
axios.get("/api/materiais/" + id)
  .then(function (resposta) {
    var material = resposta.data;
    idEditando = material.id;
    document.getElementById("nome").value = material.nome;
    document.getElementById("quantidade").value =
      material.quantidade;
  });
```

Vocês passam a usar o GET por id de verdade.

Se terminarem os três, ainda no **laboratorio**:

- Não permitir quantidade negativa no cadastro
- Mostrar o id na lista
- Depois de retirar com sucesso, limpar a mensagem de erro antiga

Checklist

Estou no projeto laboratorio?
PUT não dá add na lista?
idEditando volta para vazio depois de salvar?
GET /api/materiais/2 devolve um só?
Retirar recusa quando não há quantidade?
O catch preenche #mensagem?

Retomamos às **20h20**

Fechem o laboratório.

Parem o Spring.

Na segunda parte abrimos o **StockSales**.

Mudamos de projeto

Onde estamos

Agora é o projeto **StockSales**.
O laboratório já está encerrado.

Parem o Spring do laboratório.
Abram a pasta do StockSales e subam de novo.

Onde o StockSales parou

Em produtos, vocês já têm:

- Lista na classe do Controller
- GET, POST e DELETE
- Formulário na tela e botão Excluir

Ainda não tem PUT.

Ainda não tem clientes.

Ainda não tem venda.

O que vamos fazer no StockSales

- 1 PUT de produto — o mesmo raciocínio do laboratorio
- 2 Cadastro de cliente: GET e POST
- 3 Venda de **um** produto, com baixa de estoque

É o que acabamos de treinar, com os nomes da loja:
produto, cliente, estoque.

Produto.java — construtor vazio

Onde estamos

Projeto **StockSales**
model/Produto.java

Se ainda não tiver este construtor, acrescentem. Mantenham o outro.

```
public Produto() {  
}
```

Sem ele, POST e PUT de produto quebram.

PUT de produto

Onde estamos

controller/ProdutoController.java

```
@PutMapping("/api/produtos/{id}")
public Produto atualizar(@PathVariable int id,
                        @RequestBody Produto dados) {
    for (int i = 0; i < produtos.size(); i++) {
        if (produtos.get(i).getId() == id) {
            produtos.get(i).setNome(dados.getNome());
            produtos.get(i).setPreco(dados.getPreco());
            produtos.get(i).setEstoque(dados.getEstoque());
            return produtos.get(i);
        }
    }
    return null;
}
```

Onde estamos

```
static/js/produtos.js
```

Copiem o esquema do laboratório:

- Variável `idEditando`
- Botão Editar com `data-id`
- Submit: se o id estiver vazio → POST; se tiver id → PUT
- Preço e estoque com Number

No HTML, o form de produtos que vocês já usam no POST serve para o PUT.

Três cadastros, uma loja

Daqui a pouco teremos:

- produtos
- clientes
- vendas

Cada um pode ter o seu Controller.

As listas precisam ser as **mesmas**, senão a venda não acha o produto e o estoque não baixa.

Por isso as listas saem dos Controllers e vão para uma classe só: Dados.

Classe Dados

Onde estamos

Projeto **StockSales**

Arquivo novo:

src/main/java/com/entra21/stocksales/Dados.java

```
package com.entra21.stocksales;

import java.util.ArrayList;
import com.entra21.stocksales.model.Produto;

public class Dados {
    public static ArrayList<Produto> produtos = new ArrayList<>();
    public static int proximoIdProduto = 1;
}
```

static = uma lista só, visível para qualquer classe do projeto.

Mover a lista de produtos

Onde estamos

ProdutoController.java

Tirem os campos produtos e proximoId da classe.

O construtor passa a gravar em Dados:

```
public ProdutoController() {  
    if (Dados.produtos.isEmpty()) {  
        Dados.produtos.add(new Produto(1, "Teclado Mecanico", 250, 12));  
        Dados.produtos.add(new Produto(2, "Mouse Gamer", 120, 30));  
        Dados.produtos.add(new Produto(3, "Monitor 24", 900, 8));  
        Dados.produtos.add(new Produto(4, "Headset", 200, 15));  
        Dados.proximoIdProduto = 5;  
    }  
}
```

GET, POST, PUT e DELETE usam Dados

Onde estamos

ProdutoController.java

Troquem produtos por Dados.produtos.

Troquem proximoId por Dados.proximoIdProduto.

```
@GetMapping("/api/produtos")
public ArrayList<Produto> listarProdutos() {
    return Dados.produtos;
}
```

O if (Dados.produtos.isEmpty()) evita duplicar o seed se o Spring criar o Controller de novo.

Classe Cliente

Onde estamos

Arquivo novo:

src/main/java/com/entra21/stocksales/model/Cliente.java

```
public class Cliente {  
    private int id;  
    private String nome;  
    private String email;  
    private String telefone;  
  
    public Cliente() {}  
  
    public Cliente(int id, String nome, String email, String telefone) {  
        this.id = id;  
        this.nome = nome;  
        this.email = email;  
        this.telefone = telefone;  
    }  
}
```

Listas de cliente em Dados

Onde estamos

Dados.java, acrescentar

```
public static ArrayList<Cliente> clientes = new ArrayList<>();  
public static int proximoIdCliente = 1;
```

Import de Cliente no topo do arquivo.

Onde estamos

Arquivo novo:

controller/ClienteController.java

```
@RestController
public class ClienteController {

    public ClienteController() {
        if (Dados.clientes.isEmpty()) {
            Dados.clientes.add(new Cliente(1, "Ana", "ana@email.com", "1199"));
            Dados.clientes.add(new Cliente(2, "Bruno", "bruno@email.com", "1188"));
            Dados.clientes.add(new Cliente(3, "Carla", "carla@email.com", "1177"));
            Dados.proximoIdCliente = 4;
        }
    }

    @GetMapping("/api/clientes")
    public ArrayList<Cliente> listar() {
        return Dados.clientes;
    }
}
```

POST de cliente

Onde estamos

Ainda em `ClienteController.java`

```
@PostMapping("/api/clientes")
public Cliente cadastrar(@RequestBody Cliente cliente) {
    cliente.setId(Dados.proximoIdCliente);
    Dados.proximoIdCliente = Dados.proximoIdCliente + 1;
    Dados.clientes.add(cliente);
    return cliente;
}
```

Testem `/api/clientes` no navegador:
Ana, Bruno e Carla.

Tela de clientes

Onde estamos

Arquivos novos:

`static/clientes.html`

`static/js/clientes.js`

Copiem o desenho de produtos:

- Formulário: nome, e-mail, telefone
- Lista na `<ul>`
- `carregarClientes` com GET
- Submit com POST e `preventDefault`

Na home, um link para `/clientes.html`.

O que é uma venda nesta aula

Uma venda, hoje, tem **um** cliente e **um** produto.

- Quem compra: o cliente
- O quê: um produto
- Quantas unidades
- $\text{Total} = \text{preço} \times \text{quantidade}$

Ao confirmar: o estoque daquele produto diminui.
Se não houver estoque, a venda **não** é gravada.

Classe Venda

Onde estamos

Arquivo novo:

src/main/java/com/entra21/stocksales/model/Venda.java

```
public class Venda {  
    private int id;  
    private int clienteId;  
    private int produtoId;  
    private int quantidade;  
    private double total;  
  
    public Venda() {  
    }  
}
```

Construtor vazio + gets e sets dos cinco atributos.

Onde estamos

Dados.java

```
public static ArrayList<Venda> vendas = new ArrayList<>();  
public static int proximoIdVenda = 1;
```


O POST da venda

O JavaScript manda só isto:

```
{"clienteId": 1, "produtoId": 2, "quantidade": 3}
```

O servidor faz o resto:

- 1 Achar o cliente
- 2 Achar o produto
- 3 Se estoque < quantidade, recusar
- 4 $\text{total} = \text{preco} * \text{quantidade}$
- 5 Diminuir o estoque
- 6 Guardar a venda e devolver o JSON

Começar o VendaController

Onde estamos

Arquivo novo:

controller/VendaController.java

```
@RestController  
public class VendaController {
```

Os métodos GET e POST entram **dentro** desta classe.
O POST é longo: vamos escrever em três passos.

VendaController — achar cliente

Onde estamos

VendaController.java, método cadastrar

```
@PostMapping("/api/vendas")
public Venda cadastrar(@RequestBody Venda pedido) {
    Cliente cliente = null;
    for (int i = 0; i < Dados.clientes.size(); i++) {
        if (Dados.clientes.get(i).getId() == pedido.getClientId()) {
            cliente = Dados.clientes.get(i);
        }
    }
    if (cliente == null) {
        throw new ResponseStatusException(
            HttpStatus.BAD_REQUEST, "Cliente nao encontrado");
    }
}
```

O método ainda não acabou. Continua no próximo slide.

Onde estamos

Ainda no mesmo método cadastrar

```
Produto produto = null;
for (int i = 0; i < Dados.produtos.size(); i++) {
    if (Dados.produtos.get(i).getId() == pedido.getProdutoId()) {
        produto = Dados.produtos.get(i);
    }
}
if (produto == null) {
    throw new ResponseStatusException(
        HttpStatus.BAD_REQUEST, "Produto nao encontrado");
}
```

VendaController — estoque e gravação

Onde estamos

Ainda no mesmo método cadastrar

```
if (produto.getEstoque() < pedido.getQuantidade()) {  
    throw new ResponseStatusException(  
        HttpStatus.BAD_REQUEST, "Estoque insuficiente");  
}  
  
pedido.setId(Dados.proximoIdVenda);  
Dados.proximoIdVenda = Dados.proximoIdVenda + 1;  
pedido.setTotal(produto.getPreco() * pedido.getQuantidade());  
produto.setEstoque(  
    produto.getEstoque() - pedido.getQuantidade());  
Dados.vendas.add(pedido);  
return pedido;  
}
```

GET das vendas

Onde estamos

Ainda em VendaController.java

```
@GetMapping("/api/vendas")  
public ArrayList<Venda> listar() {  
    return Dados.vendas;  
}
```

Não esqueçam os imports: HttpStatus, ResponseStatusException, Dados, Cliente, Produto, Venda.

A tela precisa de um select

Em vez de a pessoa lembrar o id, a página mostra uma lista para escolher.

```
<select id="clienteId"></select>
```

Cada <option> tem o value = id e o texto = nome.

O JS preenche as opções com o GET de clientes e o GET de produtos.

Onde estamos

static/vendas-nova.html, arquivo novo

```
<h1>Nova venda</h1>
<a href="/">Voltar</a>
<label>Cliente</label>
<select id="clienteId"></select>
<label>Produto</label>
<select id="produtoId"></select>
<label>Quantidade</label>
<input id="quantidade" type="number">
<button id="btn-vender" type="button">Confirmar venda</button>
<p id="mensagem"></p>
<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
<script src="/js/vendas-nova.js"></script>
```


Preencher os selects

Onde estamos

static/js/vendas-nova.js, arquivo novo

```
function carregarClientes() {  
  axios.get("/api/clientes").then(function (resposta) {  
    var select = document.getElementById("clienteId");  
    select.innerHTML = "";  
    var clientes = resposta.data;  
    for (var i = 0; i < clientes.length; i++) {  
      var opcao = document.createElement("option");  
      opcao.value = clientes[i].id;  
      opcao.textContent = clientes[i].nome;  
      select.appendChild(opcao);  
    }  
  });  
}
```

O de produtos é igual, com nome e estoque no texto.

Select de produtos

Onde estamos

Ainda em vendas-nova.js

```
function carregarProdutos() {  
    axios.get("/api/produtos").then(function (resposta) {  
        var select = document.getElementById("produtoId");  
        select.innerHTML = "";  
        var produtos = resposta.data;  
        for (var i = 0; i < produtos.length; i++) {  
            var opcao = document.createElement("option");  
            opcao.value = produtos[i].id;  
            opcao.textContent = produtos[i].nome  
                + " --- estoque: " + produtos[i].estoque;  
            select.appendChild(opcao);  
        }  
    });  
}
```

Confirmar a venda — o JSON

Onde estamos

Ainda em vendas-nova.js

```
document.getElementById("btn-vender")
    .addEventListener("click", function () {
        var dados = {
            clienteId: Number(document.getElementById("clienteId").value),
            produtoId: Number(document.getElementById("produtoId").value),
            quantidade: Number(document.getElementById("quantidade").value)
        };
    });
```

Os três valores vão como número, não como texto.

Confirmar a venda — enviar

Onde estamos

Ainda no clique de Confirmar

```
axios.post("/api/vendas", dados)
  .then(function () {
    document.getElementById("mensagem").textContent =
      "Venda registrada";
    carregarProdutos();
  })
  .catch(function () {
    document.getElementById("mensagem").textContent =
      "Nao foi possivel registrar a venda";
  });
});

carregarClientes();
carregarProdutos();
```

Histórico de vendas

Onde estamos

static/vendas.html e static/js/vendas.js

Página com uma `<ul id="lista-vendas">`.

```
axios.get("/api/vendas").then(function (resposta) {  
    var vendas = resposta.data;  
    var lista = document.getElementById("lista-vendas");  
    lista.innerHTML = "";  
    for (var i = 0; i < vendas.length; i++) {  
        var li = document.createElement("li");  
        li.textContent = "Cliente " + vendas[i].clienteId  
            + " - produto " + vendas[i].produtoId  
            + " - qtd " + vendas[i].quantidade  
            + " - total R$ " + vendas[i].total;  
        lista.appendChild(li);  
    }  
});
```

Na home: links para nova venda e para o histórico.

O caminho da venda

- 1 A pessoa escolhe Ana, Mouse Gamer, quantidade 2
- 2 JS manda POST /api/vendas
- 3 Servidor acha cliente e produto nas listas de Dados
- 4 Confere estoque, calcula total, diminui estoque
- 5 A tela de produtos, ao recarregar, já mostra o estoque menor

Editar produto não duplica

/api/clientes lista Ana, Bruno e Carla

Dá para cadastrar um cliente novo

Nova venda baixa o estoque

Vender acima do estoque mostra a mensagem

Histórico lista a venda com o total

Se der tempo:

- PUT e DELETE de cliente
- No histórico, mostrar o **nome** do cliente e do produto, não só o id. Percorram as listas de Dados no JS, ou incluam os nomes na resposta da venda
- Não permitir quantidade 0 ou negativa

O que não entra hoje

- Vários produtos na mesma venda
- Cancelar venda e devolver estoque
- Banco de dados
- Login

Meta no StockSales

PUT de produto.

Clientes cadastráveis.

Uma venda de um item, com estoque baixando.

Produto pode ser alterado

Cliente aparece na lista e no select da venda

Venda grava total no servidor

Estoque diminui

Estoque insuficiente recusa a venda inteira

Home leva a produtos, clientes, nova venda e histórico

O que vimos hoje

- 1 PUT altera um item que já existe, pelo id
- 2 GET com id devolve um só
- 3 Retirar / vender é uma ação com regra
- 4 Se a regra falhar, o servidor recusa e o JS usa catch
- 5 No StockSales, as listas ficam em Dados para todo mundo ver a mesma loja

Aula 05

- Organizar o código: as listas saem do `static` de `Dados`
- Validação
- Erros HTTP com mais clareza

Tragam o `StockSales` com venda de um item funcionando.

Obrigado!

Mauricio Duailibi Neto
Turma Java Entra21